RISC-V PIPELINE VERIFICATION TEST CASES

==========

TEST CASE 1: FORWARDING PRIORITY COLLISION

==========

Goal

Force a consumer in EX to see the same source register from both EX/MEM and MEM/WB at the same time. The forwarding unit must choose EX/MEM (forwardA/B = 10) over MEM/WB (forwardA/B = 01).

Architectural Vulnerability Being Tested

This test checks forwarding priority correctness. If the forwarding selector chooses MEM/WB instead of EX/MEM during a collision, the ALU will consume a stale value and produce the wrong result.

Assembly Test Sequence

```
   addi x5, x0, 1         # I1: x5 = 1
   addi x5, x5, 2         # I2: x5 = 3
   slli x5, x5, 2         # I3: x5 = 12
   add  x6, x5, x5        # I4: must use I3's x5 = 12 on both ALU
inputs
   addi x7, x0, 0         # Spacer 1
   addi x8, x0, 0         # Spacer 2
   sw   x6, 128(x0)       # Signature store: mem[128] = 24
   beq  x0, x0, halt1
halt1:
   beq  x0, x0, halt1
```

Expected Architectural Result

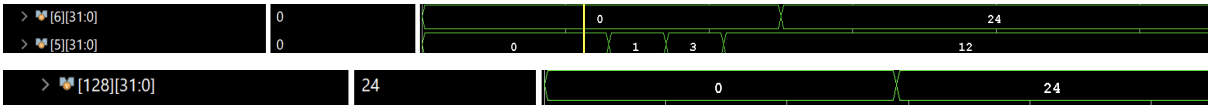
- x5 = 12
- x6 = 24
- mem[128] = 24

Cycle-by-Cycle Pipeline Analysis

1. Cycle 1
   IF = I1
   The processor fetches addi x5, x0, 1.
2. Cycle 2
   IF = I2, ID = I1
   Normal decode.
3. Cycle 3
   IF = I3, ID = I2, EX = I1
   I1 computes x5 = 1.
4. Cycle 4
   IF = I4, ID = I3, EX = I2, MEM = I1
   I2 depends on x5 from I1.
   Expected forwarding: forwardA = 10.

5. Cycle 5
   IF = I5, ID = I4, EX = I3, MEM = I2, WB = I1
   I3 depends on x5 from I2.
   Expected forwarding: forwardA = 10.
6. Cycle 6
   IF = I6, ID = I5, EX = I4, MEM = I3, WB = I2
   This is the critical collision cycle.
   I4 needs x5 on both ALU inputs.
   EX/MEM contains I3 writing x5 = 12.
   MEM/WB contains I2 writing x5 = 3.
   Expected forwarding:
   - forwardA = 10
   - forwardB = 10
   Correct ALU result: x6 = 12 + 12 = 24.
   Incorrect priority would give x6 = 3 + 3 = 6.
7. Cycle 7
   IF = I7, ID = I6, EX = I5, MEM = I4, WB = I3
   The correct x6 result is now moving down the pipeline.

Results



Correctly running

==========

TEST CASE 2: LOAD-USE HAZARD ADJACENT TO A BRANCH FLUSH

==========

Goal

Create the exact corner case where a load-use stall request and a taken branch flush happen together. The PC must still redirect to the branch target even though the hazard unit is asking to freeze the front of the pipeline.

Architectural Vulnerability Being Tested

This test checks the interaction between the load-use hazard detector and branch redirection logic. A taken branch in MEM must override the PC stall request so that the wrong path does not survive.

Assembly Test Sequence

```
   addi x3, x0, 64         # I1: base address for younger
wrong-path load
   addi x7, x0, 0          # I2: sentinel register, must remain 0
   beq  x0, x0, target2    # I3: always taken, resolved in MEM
   lw   x5, 0(x3)          # I4: in EX when I3 reaches MEM
```

```
   add  x6, x5, x0        # I5: dependent user in ID, requests
stall
   addi x7, x0, 7         # I6: wrong-path instruction that must
be flushed
target2:
   sw   x7, 132(x0)       # Must store 0 if branch wins over stall
   addi x8, x0, 42        # Marker showing target path executed
   addi x10, x0, 0        # Spacer
   sw   x8, 136(x0)       # Signature store: mem[136] = 42
   beq  x0, x0, halt2
halt2:
   beq  x0, x0, halt2
```

<u>Expected Architectural Result</u>

- x7 = 0
- x8 = 42
- mem[132] = 0
- mem[136] = 42

<u>Cycle-by-Cycle Pipeline Analysis</u>

1. Cycle 1
   IF = I1
2. Cycle 2
   IF = I2, ID = I1
3. Cycle 3
   IF = I3, ID = I2, EX = I1
4. Cycle 4
   IF = I4, ID = I3, EX = I2, MEM = I1
   The branch instruction is in decode.
5. Cycle 5
   IF = I5, ID = I4, EX = I3, MEM = I2, WB = I1
   I3 compares x0 == x0, so the branch condition is true.
6. Cycle 6
   IF = I6, ID = I5, EX = I4, MEM = I3, WB = I2
   This is the critical coincidence cycle.
   Hazard unit sees:
   - I4 is lw in EX
   - I5 uses x5 in ID
   Hazard unit requests:
   - PCWrite = 0
   - IF_ID_Write = 0
   - control_flush = 1
   At the same time, I3 reaches MEM and asserts PCSource = 1.
   Expected behavior:
   - The taken branch overrides the stall for the PC update.
   - The machine redirects to target2.
7. Cycle 7

IF = target-path fetch, ID = bubble, EX = bubble, MEM = bubble, WB = I3

The wrong-path younger instructions are flushed:

- I4 is flushed from EX/MEM
- I5 is flushed from ID/EX
- I6 is flushed from IF/ID

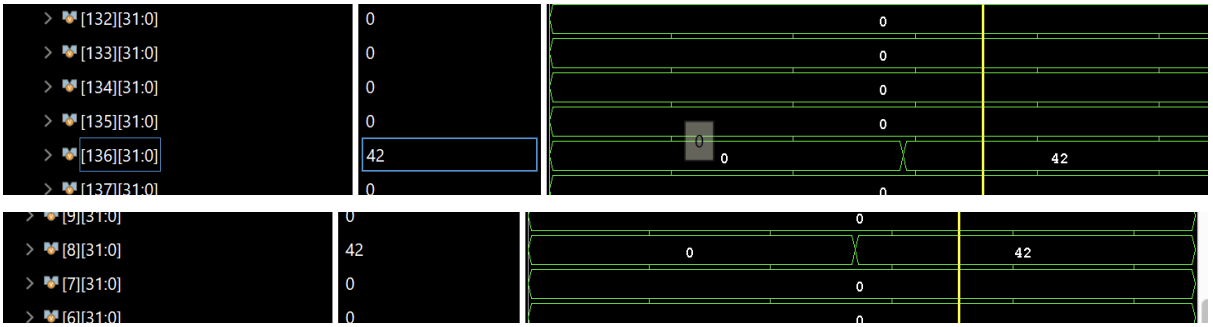
8. Cycle 8

Target-path execution begins cleanly.

Failure Signature

If the stall incorrectly blocks the branch update, the old PC can survive one extra cycle and wrong-path state may execute or partially commit.

Results



Correctly running

==========

TEST CASE 3: x0 EDGE CASES

==========

Goal

Verify that x0 is ignored by both the forwarding unit and the hazard detection unit. This test tries to write to x0, load into x0, and immediately consume x0 to make sure the pipeline treats x0 as hardwired zero at all times.

Architectural Vulnerability Being Tested

This test checks three x0-specific failure modes:

1. False forwarding from an instruction that writes x0
2. False load-use stall when the load destination is x0
3. False forwarding from MEM/WB when the destination register is x0

Assembly Test Sequence

```
   addi x1, x0, 96        # I1: base address for data word
   addi x2, x0, 99        # I2: payload
   addi x6, x0, 0         # I3: spacer before store
   sw   x2, 0(x1)         # I4: mem[96] = 99
```

```
   addi x0, x0, 7         # I5: attempted write to x0, must be
ignored
   add  x3, x0, x0        # I6: must still read 0 and 0
   lw   x0, 0(x1)         # I7: attempted load into x0, must be
ignored
   addi x4, x0, 1         # I8: must not stall, x4 must become 1
   add  x5, x0, x2        # I9: must not forward load result into
x0 source
   addi x10, x0, 0        # I10: spacer
   addi x11, x0, 0        # I11: spacer
   sw   x3, 140(x0)       # Signature store: mem[140] = 0
   sw   x4, 144(x0)       # Signature store: mem[144] = 1
   sw   x5, 148(x0)       # Signature store: mem[148] = 99
   beq  x0, x0, halt3
halt3:
   beq  x0, x0, halt3
```

<u>Expected Architectural Result</u>

- x3 = 0
- x4 = 1
- x5 = 99
- mem[140] = 0
- mem[144] = 1
- mem[148] = 99

<u>Cycle-by-Cycle Pipeline Analysis</u>

Window A: Attempted Write to x0

1. Cycle 5
   IF = I5, ID = I4, EX = I3, MEM = I2, WB = I1
   addi x0, x0, 7 enters IF.
2. Cycle 6
   IF = I6, ID = I5, EX = I4, MEM = I3, WB = I2
   I5 decodes with rd = x0.
3. Cycle 7
   IF = I7, ID = I6, EX = I5, MEM = I4, WB = I3
   I5 computes an ALU value, but architecturally it must be discarded.
4. Cycle 8
   IF = I8, ID = I7, EX = I6, MEM = I5, WB = I4
   I6 = add x3, x0, x0 is in EX while I5 is in MEM.
   Expected behavior:
   - No EX/MEM forwarding match is allowed because destination = x0.
   - x3 must become 0, not 14.

Window B: Load into x0 Followed Immediately by x0 Consumer

5. Cycle 9
   IF = I9, ID = I8, EX = I7, MEM = I6, WB = I5

This is the hazard-check cycle.

I7 = lw x0, 0(x1) is in EX.
I8 = addi x4, x0, 1 is in ID.
Expected hazard-unit outputs:
- PCWrite = 1
- IF_ID_Write = 1
- control_flush = 0
There must be no stall because the load destination is x0.

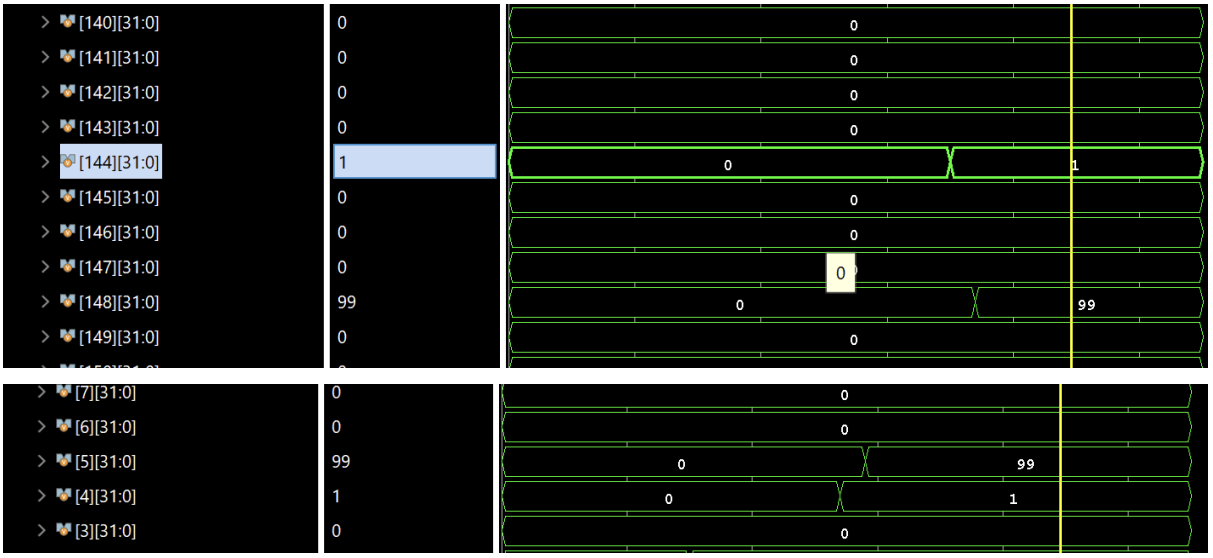
6. Cycle 10
IF = I10, ID = I9, EX = I8, MEM = I7, WB = I6
I8 advances normally and computes x4 = 1.

7. Cycle 11
IF = I11, ID = I10, EX = I9, MEM = I8, WB = I7
I9 = add x5, x0, x2 is in EX while I7 is in WB.

Expected behavior:
- No MEM/WB forwarding match is allowed because destination = x0.
- x5 must become 99, not 198.

Failure Signature
If x0 guards are missing, you may see:
- A false stall between I7 and I8
- x4 delayed by one cycle
- x5 corrupted by an illegal forward

Results



Correctly running

==========

TEST CASE 4: CONSECUTIVE DATA HAZARDS

==========

Goal
Create a long chain of back-to-back arithmetic dependencies where every instruction depends only on the immediately previous result.

This should force sustained EX/MEM forwarding across multiple consecutive cycles.

Architectural Vulnerability Being Tested

This test checks whether the forwarding unit can sustain repeated EX/MEM forwarding without accidentally selecting MEM/WB or dropping back to the stale register-file value.

Assembly Test Sequence

```
   addi x1, x0, 5          # I1: x1 = 5
   add  x2, x1, x1         # I2: x2 = 10
   addi x3, x2, 3          # I3: x3 = 13
   slli x4, x3, 1          # I4: x4 = 26
   xori x5, x4, 7          # I5: x5 = 29
   andi x6, x5, 15         # I6: x6 = 13
   add  x7, x6, x6         # I7: x7 = 26
   addi x10, x0, 0         # Spacer 1
   addi x11, x0, 0         # Spacer 2
   sw   x7, 152(x0)        # Signature store: mem[152] = 26
   beq  x0, x0, halt4
halt4:
   beq  x0, x0, halt4
```

Expected Architectural Result

- x1 = 5
- x2 = 10
- x3 = 13
- x4 = 26
- x5 = 29
- x6 = 13
- x7 = 26
- mem[152] = 26

Cycle-by-Cycle Pipeline Analysis

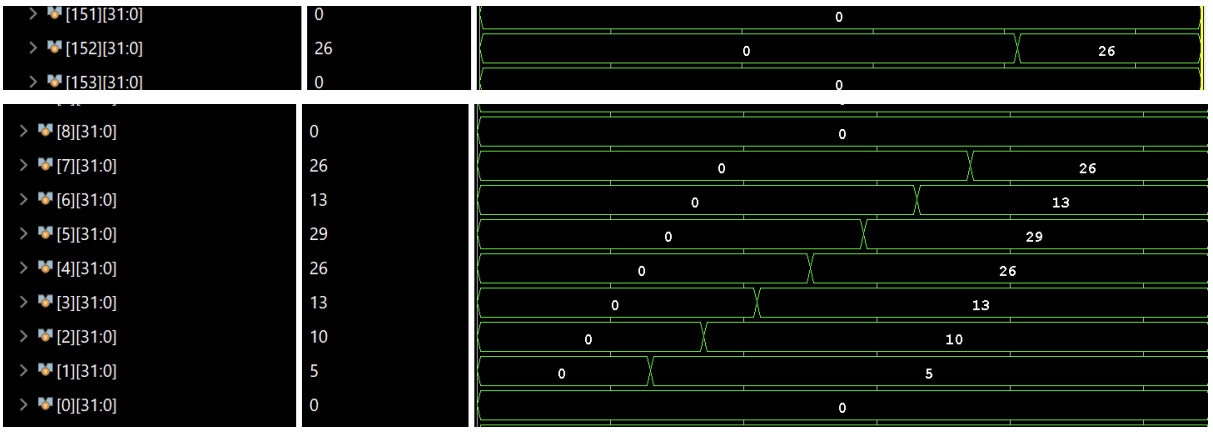
1. Cycle 1
   IF = I1
2. Cycle 2
   IF = I2, ID = I1
3. Cycle 3
   IF = I3, ID = I2, EX = I1
   I1 computes x1 = 5.
4. Cycle 4
   IF = I4, ID = I3, EX = I2, MEM = I1
   I2 needs x1 from EX/MEM on both ALU inputs.
   Expected forwarding:
   - forwardA = 10
   - forwardB = 10

5. Cycle 5
   IF = I5, ID = I4, EX = I3, MEM = I2, WB = I1
   I3 needs x2 from EX/MEM.
   Expected forwarding:
   - forwardA = 10
6. Cycle 6
   IF = I6, ID = I5, EX = I4, MEM = I3, WB = I2
   I4 needs x3 from EX/MEM.
   Expected forwarding:
   - forwardA = 10
7. Cycle 7
   IF = I7, ID = I6, EX = I5, MEM = I4, WB = I3
   I5 needs x4 from EX/MEM.
   Expected forwarding:
   - forwardA = 10
8. Cycle 8
   IF = I8, ID = I7, EX = I6, MEM = I5, WB = I4
   I6 needs x5 from EX/MEM.
   Expected forwarding:
   - forwardA = 10
9. Cycle 9
   IF = I9, ID = I8, EX = I7, MEM = I6, WB = I5
   I7 needs x6 from EX/MEM on both ALU inputs.
   Expected forwarding:
   - forwardA = 10
   - forwardB = 10

<u>Key Check</u>

Every critical consumer should take its operand from EX/MEM. If MEM/WB gets selected during the dependency chain, the comparator or priority logic is incorrect.

<u>Results</u>



Correctly running

ALL TEST CASES PASSED